

Test-checklist – Validatietabel Stageverslag

Branch: feat/stage-report-logging (v2) | Logbestand:

user://calibration_logs/calib_<timestamp>.log

0. Voorbereiding (eenmalig)

- ☐ APK met instrumentatie staat op de tablet (`UP-Studio-dev.apk` of nieuwer)
- ☐ Tablet via USB verbonden met laptop (voor logs ophalen met `adb`) — *optioneel, mag ook via bestandsmanager*
- ☐ Oude logs opgeruimd:
`adb shell "rm -f /sdcard/Android/data/com.example.upstudiogdscript/files/calibration_logs/*.log"`
- ☐ Lift volledig geassembleerd op de testrail en werkt (handmatige piloting getest)
- ☐ Laadcontact aan bovenzijde of onderzijde aangesloten (start-station gedefinieerd)

1. Per calibratierun (doe dit 3x op 3 verschillende rails)

Rail-configuraties

- ☐ **Run 1** — middellange rail, ≥ 4 horizontale bochten
- ☐ **Run 2** — korte rail (≤ 3.5 m), weinig bochten
- ☐ **Run 3** — lange rail (≥ 6 m), 6+ bochten (stress-test)

Stappen per run

1. ☐ Lift via BT verbonden, traction + spindle waardes zichtbaar in UI
2. ☐ Doorgelopen tot de calibratierun-stap in het stappenplan
3. ☐ Druk **"Start calibration"** in de app
 → logger start automatisch, sessielabel = `calibration_run`
4. ☐ **Minimaal 5 seconden stilstaan** na het indrukken van Start
 → nodig voor de stationaire jitter-metric (raw + Kalman)
5. ☐ Joystick ingedrukt houden, lift van onder naar boven laten rijden
6. ☐ Firmware stopt automatisch bij elke horizontale bocht (`STOPPED_ON_STATION`)
 → per bocht: hoekcorrectie + encoder-snapshot gelogd
7. ☐ **Tussendoor 1–2x een rechte stuk van ≥ 1 m rijden met ingedrukte joystick**
 → voor `motion_jitter_*` metric (raw vs Kalman RMSE tijdens beweging)
8. ☐ Rijd door tot `MOVE_TO_LAST` (near-end waarschuwing)
9. ☐ Override inschakelen, lift precies op laadstation positioneren
10. ☐ **Confirm-knop ingedrukt** → SUMMARY + end-station error gelogd
11. ☐ Noteer rail-label + datum/tijd + gemeten fysieke afwijking bij eindstation

Noteringen per run

	Run 1	Run 2	Run 3
Rail-label / locatie			
Datum & tijd start			
Aantal bochten verwacht			
Logbestand-timestamp			
Bijzonderheden / handelingen			

2. Optionele stress-scenario's (eenmalig, niet per run)

Doe dit in één extra run zodat er logregels zijn die de “logged events”-kolom van de verslagtabel vullen.

- ☐ **BT disconnect-test**: schakel de BT-dongle van de lift 3 seconden uit tijdens de run
→ verwacht: [EVENT] `bt_disconnect` en `bt_reconnect reconnect_ms=...`
- ☐ **Latency spike**: tablet onder zware CPU-load zetten (grote andere app starten op achtergrond)
→ verwacht: [EVENT] `latency_spike msg_id=... elapsed_ms=...`
- ☐ **Manual compass re-trigger**: compass-reset-knop in UI tijdens een straight-section indrukken
→ verwacht: [COMPASS] `reset reason=...`
- ☐ **Split-bend**: indien aanwezig, pak een rail met 2x horizontale bocht van zelfde richting < 250 mm uit elkaar
→ verwacht: [EVENT] `split_bend_armed ...`

3. Logs ophalen en verwerken

- ☐ Tablet via USB aansluiten, USB-debugging aan
- ☐ Log-folder ophalen:
`adb pull /sdcard/Android/data/com.example.upstudiogdscript/files/calibration_logs/ ~/stageverslag_pe`
- ☐ Per logbestand: `grep "[SUMMARY]"calib_*.log` → geeft de getallen voor de verslagtabel
- ☐ Cijfers invullen in `main.tex` Tabel “Validation measurements” (regels 1222–1238)
- ☐ Events (BT disconnect / latency spike / compass re-trigger) tekstueel in de “logged events”-kolom

4. Desktop frame-rate meting (voor verslagtabel-rij “Desktop FPS”)

- ☐ Build de Linux-export (`make linux` of via Godot-editor)
→ `export/linux/UP-Studio.x86_64` of vergelijkbaar
- ☐ Start de app op de laptop, verbind met een lift via TCP (of een mock-transport)
- ☐ Doorloop één calibratierun — de FPS-sampler schrijft [FPS] `fps=... platform=linux` weg
- ☐ Haal de avg/min/max uit de SUMMARY-regels voor de desktop-kolom

Notities onderaan
